

ONCFM

Détection automatique de faux billets

Analyse exploratoire · Modélisation · Pipeline de prédiction

Portfolio IA · Data Analyst · Août 2026

01

Contexte & Objectif



Contexte & Objectif

Mission

Détecter automatiquement les faux billets en euros

À partir de 6 mesures géométriques (en mm)

1 500 billets fournis, scannés et labellisés

Objectif final : déploiement en production (ONCFM)

6 variables géométriques

length : longueur du billet

height_left / height_right : hauteurs gauche/droite

margin_up / margin_low : marges supérieure/inférieure

diagonal : diagonale du billet

02

Exploration des données



Exploration des données — Résultats clés

1 500 billets : 1 000 vrais (66,7%) et 500 faux (33,3%) → dataset déséquilibré, **accuracy** trompeuse

37 valeurs manquantes sur **margin_low** (2,5%) → imputation nécessaire avant modélisation

Variables les plus corrélées à **is_genuine** : **length** ($r=0.85$), **margin_low** (-0.78), **margin_up** (-0.61)

diagonal faiblement corrélée ($r=0.13$) → peu discriminante entre vrais et faux billets

length et **margin_low** : distributions quasi disjointes entre classes → variables clés pour la prédiction

03

Prétraitement



Prétraitement — Méthodologie rigoureuse

- ▶ Split train/test avant toute modélisation pour évaluer les modèles sur des données jamais vues
- ▶ Séparation : 80% train (1 200 billets) / 20% test (300 billets) — stratifiée (même proportion 66%/33%)
- ▶ 37 valeurs manquantes sur **margin_low** : Imputation **KNNImputer** (k=5) : fit uniquement sur **X_train**, **transform** sur **X_test** — pas de fuite
- ▶ Standardisation (**StandardScaler**) pour la régression logistique, le KNN et le K-means
- ▶ Random Forest : aucune standardisation nécessaire car le modèle repose sur des arbres de décision

04

Modélisation



4 modèles testés et comparés

Petit rappel sur les métriques de classification :

- VP = vrais positifs (vrai billet prédit comme vrai)
- VN = vrais négatifs (vrai billet prédit comme faux)
- FP = faux positifs (faux billet prédit comme vrai cas strictement interdit)
- FN = faux négatifs (faux billet prédit comme faux)

Accuracy: Part des billets bien classés au total.
 $Accuracy = (VP + VN) / (VP + VN + FP + FN)$

Précision: Parmi les billets prédits comme faux, part de ceux qui sont vraiment faux.
 $Précision = VP / (VP + FP)$

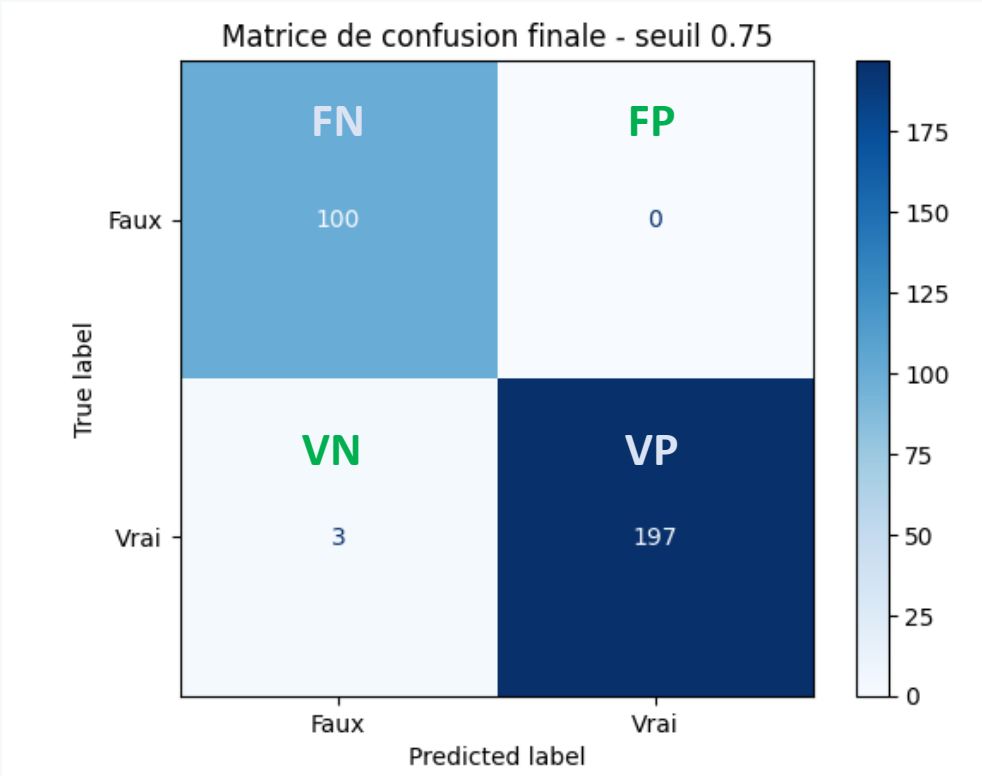
Recall: Parmi les vrais faux billets, part de ceux qu'on détecte bien.
 $Recall = VP / (VP + FN)$

F1-score: Moyenne harmonique entre précision et recall.
 $F1-score = 2 \times (Précision \times Recall) / (Précision + Recall)$

F-beta score: Version du F1 qui donne plus de poids au recall si $\beta > 1$.
 $F\beta = (1 + \beta^2) \times (Précision \times Recall) / ((\beta^2 \times Précision) + Recall)$

Avec $\beta = 2$, on privilégie davantage le recall.

Le ROC AUC mesure la capacité globale du modèle à séparer les vrais et les faux billets, quel que soit le seuil; il n'est pas disponible sur k-means car c'est un modèle non supervisé.



	Modèle	Accuracy	Précision (Faux)	Recall (Faux)	F1-score (Faux)	ROC AUC
0	Régression logistique	0.990000	0.989899	0.98	0.984925	0.99955
1	Random Forest	0.990000	0.989899	0.98	0.984925	0.99930
2	K-means	0.986667	0.980000	0.98	0.980000	NaN
3	KNN	0.983333	0.979798	0.97	0.974874	0.99720

4 modèles testés et comparés

99%

Régression Logistique

Modèle linéaire simple et interprétable.

Recall faux : 0.98. Très bon compromis global.

98%

KNN (k=5)

Vote des 5 voisins les plus proches.

Recall faux : 0.97. Correct mais un peu moins précis.

99%

Random Forest

Ensemble d'arbres de décision.

Recall faux : 0.98. Solide mais légèrement derrière ici.

99%

K-means (non sup.)

Méthode non supervisée. Les clusters sont ensuite rattachés aux classes.

Recall faux : 0.98.

Comparaison des 4 modèles — Recall sur les faux billets

Métrique prioritaire : recall sur les faux billets (objectif = détecter 100% des faux)

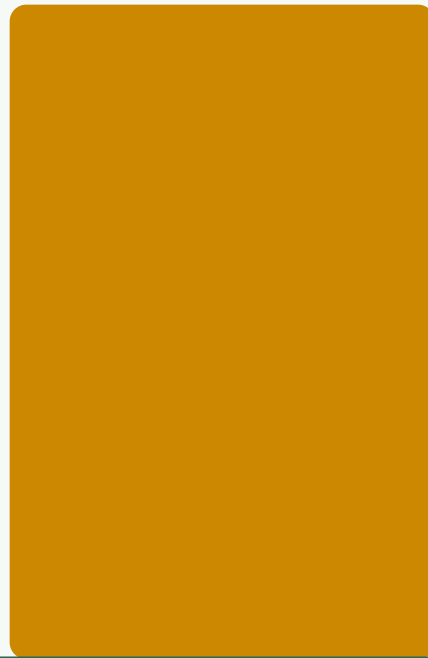
98%



Régression
Logistique

Acc: 99.0%

97%



KNN

Acc: 98.7%

98%



Random
Forest

Acc: 99%

98%



K-means

Acc: 98.7%

05

Choix & Optimisation



Pourquoi la régression logistique ?

- ▶ Performances égales à **Random Forest** : **accuracy** 99% et **recall faux** de 0.989 au seuil standard

- ▶ Modèle simple à expliquer et facile à intégrer dans un pipeline de prédiction

- ▶ Les probabilités de sortie permettent d'ajuster le seuil selon le besoin métier

- ▶ Avec un seuil ajusté à 0.75, on ne laisse passer aucun faux billet dans notre test

- ▶ Le modèle reste léger, rapide à exécuter et cohérent avec les objectifs de la mission

Ajustement du seuil de décision

Seuil par défaut (0.5) : 2 faux billets classés comme vrais — inacceptable pour l'ONCFM

Objectif métier : 0 faux billet non détecté → plusieurs seuils testés de 0.50 à 0.95

Seuil 0.75 : **recall faux** = 1.0, **précision faux** = 0.962 et aucun faux billet laissé passer

Contrepartie : 3 vrais billets rejetés à tort sur 300 (1%) → coût acceptable au regard du risque métier

Les seuils 0.70 et 0.75 donnent le même résultat sur ce test ; 0.75 a été retenu pour rester plus strict

Matrice de confusion — Régression logistique (seuil 0.75)

	Prédit Faux	Prédit Vrai	
Réel Faux	100 Vrais Négatifs (Faux → Faux)	0 ✓ Faux Négatifs (Faux → Vrai)	Résultat clé 0 faux billet non détecté Seuil 0.75 → recall faux = 1.0 Précision faux = 0.989 Contrepartie : 3 vrais billets rejetés à tort sur 300
Réel Vrai	3 Faux Positifs (Vrai → Faux)	197 Vrais Positifs (Vrai → Vrai)	

06

Pipeline & Application



Architecture du script de prédiction

1

**Lecture
des données**

CSV avec détection
automatique du séparateur
ou saisie manuelle

2

**Vérification
des colonnes et des lignes
vides**

6 variables
géométriques
Attendues, détection des lignes
vides

3

**Pipeline
sklearn**

Imputation avec les 5 voisins
+ standardisation
+ régression logistique

4

**Probabilités
de sortie**

Comparaison au seuil
choisi
pour décider vrai/faux

5

**Résultat
final**

Output binaire
vrai / faux
+ probabilités

Sortie : prédiction billet par billet + récapitulatif global + export CSV si besoin

Seuil 0.75 embarqué · Modèle sauvegardé via joblib · Paramètre --seuil disponible

Démonstration du script

Mode CSV

```
py -3.14 predict_billets.py --csv billets_production.csv  
--seuil 0.75
```

Modèle chargé avec succès.

A_1 : faux (proba_vrai=0.000967, proba_faux=0.999033)

A_4 : vrai (proba_vrai=0.981371, proba_faux=0.018629)

--- Récapitulatif ---

Total : 5 | Vrais : 2 | Faux : 3

Permet d'analyser rapidement un lot de billets au format CSV.

Mode saisie manuelle

```
py -3.14 predict_billets.py --diagonal 171.32 --height-  
left 104.86 --height-right 104.95 --margin-low 4.52 --  
margin-up 2.89 --length 112.83 --seuil 0.75
```

diagonal : 171.32

height_left : 104.86

...

Billet 1 : faux (proba_vrai=0.640663,
proba_faux=0.359337)

Permet de tester un billet isolé avec les mesures saisies au clavier.

07

Conclusion



Conclusion & Recommandations

📄 Modèle retenu : Régression logistique — simple, performante et facile à expliquer

🎯 Seuil ajusté à 0.75 → 0 faux billet classé comme vrai (priorité métier respectée)

📊 Recall faux = 1.0 · Accuracy = 99% · 3 vrais billets rejetés à tort sur 300

🔧 Pipeline professionnel : validation colonnes, imputation KNN, seuil embarqué

🚀 Application fonctionnelle : entrée CSV ou manuelle, output binaire vrai/faux et export des résultats